Track Record: Technical Report

Capstone: *Closing the Metadata Gap* (African Leadership University). Repository: github.com/SibahleD/artist-rights-registry. Report revised 31 October 2026 against the repository snapshot *artist-rights-registry-main.zip*.

Status reflects the code in that snapshot. Evidence points to files; paths are relative to the repo root. Test results are labelled as run, skipped or not run, and the environment limits are stated in section 3.

1. Scope alignment and feature validation

Planned scope comes from the project README roadmap and the approved proposal themes (complete release metadata, royalty splits, distributor hand-off).

Planned feature	Intended purpose	Status	Evidence
Accounts and auth	Secure access for each artist	Completed, extended with email verification, password reset, email change, data export and deletion, terms acceptance	authController.js, accountController.js, services/passwordReset.js, verifyEmail.js, emailChange.js; account.test.js, password-reset.test.js; migrations 002, 013, 014
Artist profile	Names, links, avatar, societies	Completed. Artist name, legal name and username are unique (case-insensitive)	userController.js, societyController.js; links.test.js, societies.test.js, uniqueNames.test.js; migrations 009, 012, 018
Releases and tracks	Capture release and song metadata (ISRC, UPC, credits, copyright)	Completed, extended with featured artists, release credits, cover versions, label and language, distribution fields	releaseController.js, trackController.js, contributorController.js; client/src/pages/ListingForm.tsx; upc.test.js; migrations 005, 010, 017, 020
Royalty splits	Per-song ownership that totals exactly 100%	Completed	server/src/lib/splits.js; splits.test.js (16 cases)
Readiness check and draft/ready lock	Stop incomplete releases reaching a distributor	Completed, extended with warnings, spelling conflicts and distributor-specific checks	services/readiness.js, services/lifecycle.js, lib/distribution.js; ready-rule, readiness, spelling, lifecycle and distribution tests
Collaborations (matched by email)	Co-owners see their shares	Completed	sharedTracks route; client/src/pages/SharedTrackView.tsx
Split disputes (accept, counter, comment)	Resolve disagreements about shares	Changed. Delivered as co-writer notices: an emailed one-time link lets a co-writer confirm or dispute without an account. The full dispute tables (disputes, dispute_splits, dispute_responses) still exist in the schema but the app does not write to them; releaseController.js reads disputes once	services/notices.js, cowriterNotice.js, publicNoticeController.js; migrations 003, 007
Cover artwork and profile pictures	Required by distributors	Completed with type, size and pixel checks. Images are limited to 4 MB because they pass through the Netlify Function	imageController.js, services/mediaInspect.js; migration 023
Audio upload	Prove each song has a master file	Completed. MP3 and FLAC only. In production the browser uploads straight to a private Supabase Storage bucket through a signed URL, then the server reads the file back and checks format, duration and bitrate. On a developer machine it falls back to local disk	audioFileController.js (audioUploadUrl, completeAudio), services/storage.js, client/src/api.ts (uploadAudio); audioDuration.test.js
Settings page	Username, password, deletion	Completed	client/src/pages/Settings.tsx
Distributor export formats	Hand metadata over without re-typing	Completed as JSON export plus a Chrome extension (RouteNote layout, manual fill only, UPC capture)	services/exportRelease.js; extension/; export.test.js, upc.test.js

Planned feature	Intended purpose	Status	Evidence
Added: playlist fit	Help artists pitch a song	Changed since the last report. The external audio-analysis service was dropped (migration 022). The artist now sets nine feature sliders and the app scores the song against nine playlist types with a rule-based estimate	lib/playlistFit.js, audioFeatureController.js, client/src/components/performance.tsx; playlistFit.test.js (6 cases)
Added: stream import and earnings	Understand performance and expected income	Completed (estimates, labelled as such). CSV import, performance by track, platform and month, exports	lib/streams.js, lib/streamExport.js, client/src/lib/stats.ts, client/src/data/rates.ts; streams*.test.js
Added: glossary, Rwanda guide, landing site	Teach release requirements	Completed. Glossary has per-user progress	glossaryController.js, client/src/pages/Glossary.tsx, client/landing/; migration 011
Production deployment	Public, usable service	Largely complete in code: Netlify (static site and API function), Supabase (PostgreSQL and file storage), hourly notice sweep, API keep-alive workflow. Not confirmed live: the README still has placeholders for the live URL, walkthrough video and report link, and no live test run is recorded	netlify.toml, netlify/functions/, .github/workflows/keep-alive.yml, services/storage.js

Scope changes. Disputes were narrowed to the notice flow so co-writers who are not yet users can still confirm a share. Streams, earnings and playlist fit were added to help artists after release and are labelled as estimates. Since the previous report, audio analysis moved from a service to manual inputs, and audio storage moved from local disk to Supabase Storage, which closes the deployment gap the earlier report listed.

2. Technical functionality

Architecture. A React 19 + TypeScript single-page app (served under /app) talks JSON over HTTPS to an Express 5 API with a PostgreSQL connection pool. Requests pass through rate limiting and JWT middleware, then route, controller, service and pure library layers. 23 SQL migrations define 21 tables with CHECK constraints, so rules hold even if the API is bypassed. The API runs as a Netlify Function in production. The codebase is about 13,000 lines across client and server source.

Frontend and backend interaction. client/src/api.ts wraps fetch: it attaches the JWT, turns failures into a typed ApiError (message plus a list of problems), redirects on 401, handles multipart uploads for images, and runs the two-step signed-URL upload for audio. Pages render the server's problem lists next to the field that needs fixing.

Data handling. Passwords use bcrypt. Emails are looked up by a keyed hash. Reset, verification and co-writer links carry a 256-bit random token and only its SHA-256 hash is stored. Uploads are inspected by content, not extension. Audio completion is tied to the signed key with a server-side ticket compared before anything is saved. Client input is never trusted: stream imports are re-validated row by row on the server.

Behaviour with different inputs

Input	Expected system response	Where checked
Splits 33.33 + 33.33 + 33.33	Rejected: totals 99.99%, message states the required total	lib/splits.js, splits.test.js
Percent with 3 decimals, negative, over 100, NaN	Rejected with the collaborator's row number	splits.test.js
Same person twice on a track	Rejected: asks to combine their share into one entry	lib/splits.js
More than 50 collaborators or none	Rejected: list must hold 1 to 50 entries	lib/splits.js
CSV row with bad ISRC, future month or negative streams	Row skipped with a reason, rest of file imported (first 25 reasons shown)	lib/streams.js, streams.test.js
Rows that share ISRC, platform and month	Added together into one row	streams.test.js
Over 20,000 import rows	Refused with advice to split the file by date range	lib/streams.js
Renamed .txt uploaded as audio	Rejected: only real MP3 or FLAC accepted. The route also refuses any file name not ending in .mp3 or .flac before issuing an upload URL	mediaInspect.js, audioFileController.js
Audio larger than the limit (50 MB default)	HTTP 413 naming the limit, before any upload URL is issued	audioFileController.js

Input	Expected system response	Where checked
Taken artist name, legal name or username	HTTP 409, comparison ignores case and spaces	uniqueNames.test.js
Editing a ready release	HTTP 409: revert to draft first	lifecycle.test.js
Reusing a password-reset link	Rejected: token is one-time and expires	password-reset.test.js
Too many requests	HTTP 429 from the rate limiter (per visitor, 15-minute window; per-user API default 300)	middleware/rateLimits.js, rateLimits.test.js

3. Testing strategies and results

The repository holds 186 server test cases in 30 files (the README count; a text search finds 184, and the other 2 are generated in loops, which running the suite confirmed: 175 loaded plus 11 in suites that could not load). The extension has 14 named tests in three files plus two script-style suites (popup-values, popup-ui) that print an all-checks-passed line.

How this revision was run. The authoring sandbox has no network, so npm packages could not be installed. To load the code, a minimal stand-in for the pg module was placed outside the repo and image-size was borrowed from another install. Suites that call express, jsdom or a real database could not run. Nothing in the repository was changed.

Strategy	Why chosen	How conducted	Result and follow-up
Unit testing of pure logic (splits, codes, streams, playlist fit, account, tokens, UPC check digit, societies, spelling, export shape, distribution rules)	These functions decide whether money and ownership data are right, so they need exhaustive edge cases	node --test; validators return {value} or {error}, so no mocks are needed. Node 22.22	175 of 186 cases loaded in this run: 120 passed, 51 skipped (database), 4 failed, see below
Integration testing against PostgreSQL (account-db, streams-db, stream-stats-db, streams-export-db, checks, lifecycle, uniqueNames)	Constraints and transactions only show their faults against a real database	Same runner with TEST_DATABASE_URL; each suite builds and tears down its own rows	**Not run.** 51 cases skipped (no database). Run npm test with the test database and record the summary
Security and abuse testing (rate limits, token reuse, hashed email lookup, mail guard)	Public registration, reset links and notice links are attack surface	Unit tests of limiter configuration and token services; production start-up guards for mail and APP_URL	tokens, emailhash, mailer, optionalMail and password-reset suites pass. rateLimits.test.js (7 cases) not run: needs express
Extension tests	The extension edits another site's form, so selectors and labels must be checked	extension/test; label parity and RouteNote label tests; popup value builder; jsdom form and popup tests	Run and passed: languages-parity (2), routenote-labels (5), popup-values. Not run: filler (7) and popup-ui, which need jsdom. The real RouteNote form is still unverified

Failures and cases not run (all environmental)

Item	Count	Cause
audioDuration.test.js: read length from file; refuse file with no usable length	2 failed	music-metadata is not installed. mediaInspect.js catches the failed import and returns its generic unreadable-audio error, so the length assertions fail. Expected to pass once npm install has run
rateLimits.test.js, audioFeatureController.test.js	11 cases not loaded	Cannot find module express
Database suites	51 skipped	TEST_DATABASE_URL not set
extension filler and popup-ui tests	not run	jsdom is not a project dependency

No test failure points at a defect in the application logic. The client has no component or end-to-end tests, and no browser run was done for this revision.

4. Performance and compatibility

Server-side algorithm timings were re-measured on Node 22.22 in the authoring sandbox (average over many calls, single machine, no load):

Operation	Input size	Measured time
parseCollaborators (splits validation)	50 collaborators (the maximum)	0.02 ms per call
projections (playlist fit, 9 playlist types)	9 audio features	0.006 ms per call
parseImport (CSV validation and merge)	20,000 rows (the maximum) merged to 4,500	about 32 ms total (average of 10 runs; fastest 18 ms)

The API is not the bottleneck at these sizes. Costs that matter are database round trips and file handling. Audio no longer passes through the API in production, because the browser uploads to the bucket directly, which avoids the Netlify Function's roughly 6 MB request limit. Images (4 MB cap) still go through the function. The previous report's in-browser 30-second clip is no longer part of the app.

Environment matrix (manual checklist)

Environment	What to check	Observed
Chrome, Edge (current)	Login, release form, audio upload to Supabase, CSV import, PDF export, extension fill	Not yet recorded
Firefox, Safari (current)	Same flows without the extension; password show/hide icon; rounded buttons	Not yet recorded
Windows 11, macOS, Android phone	Layout at 360 px, 768 px and 1280 px; upload from phone storage; mobile bottom menu	Not yet recorded
Low-end laptop or phone	Time to interactive on /app (Lighthouse)	Not yet recorded
Slow network (DevTools Slow 3G)	Error message when the API is unreachable; 4 MB CSV import; 50 MB audio upload	Not yet recorded

These cells need a person at a browser; they are left as not recorded rather than filled with guesses.

5. Code quality and engineering practice

Separation of concerns. Server: routes, controllers, services and I/O-free lib modules. Client: pages, components, lib (pure logic with typed inputs), data. Reusable components include SplitsEditor, TrackEditor, UnsavedGuard and the shared editors in editors.tsx and form.tsx.

OOP principles. The stack is mostly functional and module-based, so the principles apply in that form. Encapsulation: each module exports a small public surface (lib/splits.js exports only ROLES and parseCollaborators). Abstraction: services/storage.js hides local disk and Supabase behind one interface (it now has both, switched by environment variables, which proves the point). Inheritance: ApiError extends Error and carries problem details. Polymorphism: validators share one contract, and the mailer chooses a console or SMTP sender behind a single sendMail. Open/closed: playlist types and distributor rules are data tables, so adding one does not change the scoring code.

Algorithms. Integer-cent arithmetic for splits; Levenshtein edit distance with Unicode normalisation for name conflicts (O(mn) time, O(n) memory); hash-map merge for imports (O(n)); piecewise-linear weighted scoring for playlist fit; token hashing; check-digit validation for UPC; a three-estimate (low, typical, high) earnings model.

Refactoring done. Rate limiting lives in middleware/rateLimits.js and mailing in services/mailer.js, and every import points at those. The row editors share a generic editor, the upload size caps are defined once in limits.js, and the audio feature source was switched with a non-destructive migration (022), keeping old rows.

Remaining debt (verified in this snapshot).

Item	Detail
Stale duplicate modules	server/src/mailer.js, server/src/rateLimits.js and server/src/pool.js differ from the live copies (services/mailer.js, middleware/rateLimits.js, db/pool.js) and nothing requires them. Delete them. This undoes the claim in the earlier report that the modules were merged
Stray file in the repo tree	server/uploads/audio holds a sample .mp3. The uploads folder is gitignored, so it is only in the zip, but remove it before submitting
Largest page	client/src/pages/ListingForm.tsx is 547 lines (was 535) and should be split into section components
No client tests	The client has no component or end-to-end tests
Format list mismatch	mediaInspect.js still recognises WAV and AAC and its error text says so; the allowlist and the upload route accept only MP3 and FLAC. Align the message

Item	Detail
Stale documentation	README says 21 migrations (there are 23) and still describes audio analysis from a clip. netlify.toml points to a DEPLOY.md that is not in the repo. keep-alive.yml mentions a Render address although hosting is Netlify. README link placeholders (video, report, live URL) are unfilled
Unused schema	disputes, dispute_splits and dispute_responses tables are not written to by the app

Known gaps. Cover art and avatars stay in Postgres and are capped at 4 MB by the Netlify request limit. Supabase needs the bucket created and its file size limit raised to match MAX_AUDIO_BYTES; this has not been verified against the live project. The extension's RouteNote selectors were tested only against a local test form.